

Documentação — Implantação no Kubernetes (Minikube)

Disciplina: DevOps — **Aluno:** Lucas de Oliveira Rodrigues Alves **Aplicação:** Plataforma de Consultoria

Este documento descreve (a) a aplicação e seus componentes, (b) o roteiro de testes e (c) os artefatos Kubernetes utilizados na implantação no Minikube.

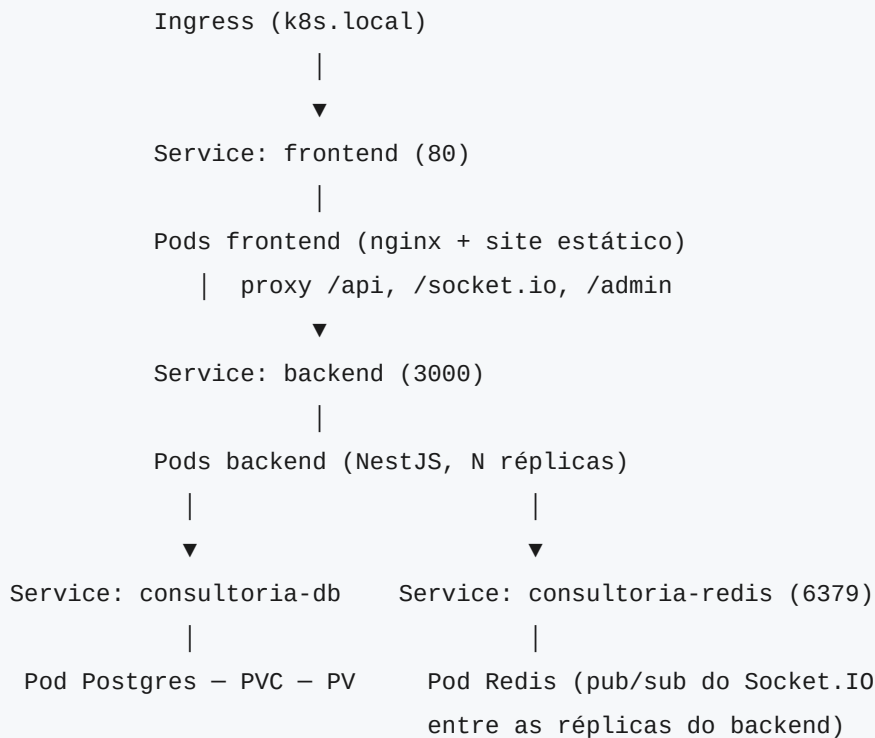
(a) A aplicação e seus componentes

A aplicação é uma **plataforma de consultoria** onde clientes fazem perguntas e consultores as respondem, com atualização em tempo real. É a mesma aplicação containerizada no trabalho anterior (Docker Compose), agora implantada no Kubernetes.

Componentes / containers

Componente	Imagem	Tecnologia	Porta	Função
Frontend	lucasoliveiraalves/consultoria-frontend	Next.js (export estático) + nginx	80	Serve a interface web e faz proxy reverso de /api, /socket.io e /admin para o backend
Backend	lucasoliveiraalves/consultoria-backend	NestJS + TypeORM + JWT + AdminJS	3000	API REST, autenticação, WebSocket e painel administrativo
Banco	postgres:16	PostgreSQL	5432	Persistência dos dados (usuários, perguntas, respostas)
Redis	redis:7-alpine	Redis	6379	Pub/sub para o adapter do Socket.IO — permite que os eventos em tempo real cheguem aos clientes mesmo com o backend escalado em várias réplicas

Arquitetura no cluster



- Apenas o **frontend** é exposto externamente, via **Ingress** (`k8s.local`).
- **Backend** e **banco** usam Services do tipo `ClusterIP` (acessíveis só dentro do cluster).
- O **banco roda dentro do cluster** (requisito do trabalho), com volume persistente.
Observação: em produção o recomendado seria um banco gerenciado/externo ou um `StatefulSet` ; aqui ele roda no cluster apenas para atender ao requisito.

(b) Roteiro de testes

Pré-requisitos

- Minikube, kubectl, Helm 3+ e Docker instalados.

Passo a passo

```
# 1. Inicia o cluster, habilita o Ingress, faz build+push das imagens e instala via Helm
# (-i inicia o minikube, -b builda o backend, -f builda o frontend)
./helm-up -ibf

# 2. Configura o acesso pelo host do Ingress (uma única vez)
echo "$(minikube ip) k8s.local" | sudo tee -a /etc/hosts

# 3. Verifica se os pods estão prontos (todos 1/1 Running)
kubectl get pods
```

Nas execuções seguintes, sem precisar rebuildar as imagens, basta `./helm-up`.

Verificação automatizada (via Ingress)

```
IP=$(minikube ip)

# Frontend deve responder 200
curl -s -o /dev/null -w "frontend HTTP %{http_code}\n" \
  -H "Host: k8s.local" http://$IP/

# Cadastro de um usuário (cria registro no banco dentro do cluster)
curl -s -H "Host: k8s.local" -H "Content-Type: application/json" \
  -X POST -d '{"name":"Teste","email":"teste@exemplo.com","password":"teste123"}' \
  http://$IP/api/auth/register

# Login (deve retornar um accessToken JWT)
curl -s -H "Host: k8s.local" -H "Content-Type: application/json" \
  -X POST -d '{"email":"teste@exemplo.com","password":"teste123"}' \
  http://$IP/api/auth/login
```

Teste pela interface

1. Acesse `http://k8s.local` no navegador.
2. Cadastre um usuário ou use um dos perfis de exemplo (`cliente@exemplo.com` / `consultor@exemplo.com`, senha `teste123`), caso a base esteja populada.

Encerramento

```
./helm-down          # remove o release Helm
# (opcional) minikube delete  # destrói o cluster
```

(c) Artefatos Kubernetes utilizados

Todos são gerados pelo Helm Chart (`k8s/consultoria/`). Lista de objetos criados:

Objeto	Nome	Papel na implantação
Deployment	<code>consultoria-db</code>	Pod do PostgreSQL; estratégia <code>Recreate</code> (evita dois pods no mesmo volume RWO)
Deployment	<code>backend</code>	Pod da API NestJS; <code>initContainer</code> (<code>wait-for-db</code>) espera o banco aceitar conexões antes de subir
Deployment	<code>frontend</code>	Pod nginx servindo o site estático
Service	<code>consultoria-db</code> (ClusterIP, 5432)	Nome DNS interno do banco, usado pelo backend em <code>POSTGRES_HOST</code>
Service	<code>backend</code> (ClusterIP, 3000)	Alvo do proxy reverso do nginx
Service	<code>frontend</code> (ClusterIP, 80)	Alvo do Ingress
ConfigMap	<code>consultoria-config</code>	Variáveis não sensíveis (host/porta do banco, nome do BD, porta e modo do backend)
Secret	<code>consultoria-secret</code>	Dados sensíveis: usuário/senha do banco, <code>JWT_SECRET</code> e credenciais de e-mail
PersistentVolume	<code>consultoria-db-pv</code>	Armazenamento do Postgres (<code>hostPath</code> dentro do Minikube, 2Gi)
PersistentVolumeClaim	<code>consultoria-db-pvc</code>	Reivindica o volume para o pod do banco
Deployment	<code>consultoria-redis</code>	Pod do Redis (pub/sub do Socket.IO)
Service	<code>consultoria-redis</code> (ClusterIP, 6379)	Nome DNS interno do Redis, usado pelo backend
Ingress	<code>consultoria-ingress</code>	Publica o frontend no host <code>k8s.local</code>

Tempo real (WebSocket) com o backend escalado

O backend usa Socket.IO. Como o Socket.IO é *stateful*, rodar várias réplicas exige dois cuidados, ambos já implementados:

1. **Adapter Redis** (`@socket.io/redis-adapter`): cada réplica do backend publica/ assina eventos no Redis, de modo que um `emit` feito por um pod chega aos clientes conectados em qualquer outro pod.
2. **Transporte WebSocket no cliente** (`io({ transports: ["websocket"] })`): evita o handshake de long-polling em múltiplas requisições (que cairia em pods diferentes sem

sticky session). A conexão vira um único WebSocket persistente para um pod.

Com isso, `backend.replicaCount` pode ser aumentado livremente no `values.yaml`.

Como os artefatos se conectam

- O **Deployment do backend** consome o `ConfigMap` e o `Secret` via `envFrom`, obtendo as variáveis de ambiente (incluindo as credenciais do banco).
- O **Deployment do banco** lê usuário/senha do `Secret` e o nome do BD do `ConfigMap`, e monta o `PVC` (ligado ao `PV`) em `/var/lib/postgresql/data` para persistir os dados.
- O **Ingress** roteia todo o tráfego de `k8s.local` para o `Service frontend`; o `nginx`, por sua vez, encaminha `/api`, `/socket.io` e `/admin` para o `Service backend`.

Estrutura do Helm Chart

```
k8s/consultoria/
├─ Chart.yaml           # chart pai (declara os 3 subcharts)
├─ values.yaml          # valores globais + overrides de cada subchart
├─ templates/
│   ├─ configmap.yaml   # ConfigMap compartilhado
│   ├─ secret.yaml      # Secret compartilhado
│   └─ ingress.yaml     # Ingress
└─ charts/
    ├─ db/              # PV, PVC, Deployment e Service do Postgres
    ├─ redis/           # Deployment e Service do Redis
    ├─ backend/         # Deployment (com initContainers) e Service da API
    └─ frontend/        # Deployment e Service do nginx
```

Scripts de automação

- `helm-up [-i] [-b] [-f]` — inicia o Minikube (`-i`), builda+publica a imagem do backend (`-b`) e/ou do frontend (`-f`) no Docker Hub (e `minikube image load`), e instala o chart via Helm.
- `helm-down` — desinstala o release Helm.
- Alternativamente, há um `Makefile` com os alvos `start`, `build`, `push`, `deploy`, `up`, `down`, `status`, `logs-backend`, `logs-db`, `hosts`.